

Architecting Scalable MVP Applications on Wix: A Framework for AI-Assisted Development with Velo

Introduction

Executive Summary

This report presents a strategic architectural framework for developing a standardized, scalable Minimum Viable Product (MVP) application template on the Wix platform. The objective is to establish a "1st day standard template" that is not only functional, efficient, and secure but also serves as a robust, extensible model for future development across multiple projects. The framework is designed for technical founders and lead developers seeking to build professional-grade applications by leveraging the full capabilities of Wix's development environment, Velo, in conjunction with modern, AI-assisted workflows. The resulting blueprint provides a replicable methodology for creating applications that are responsive across all devices, maintain persistent user sessions, and are architected for long-term performance and maintainability.

The Paradigm Shift: From Velo to "Vibe Coding"

An initial analysis of the development landscape, prompted by a review of influential content such as Tina Huang's "Vibe Coding Fundamentals In 33 minutes," reveals a significant paradigm shift in web development.¹ The task of building a modern web application is no longer confined to the traditional, line-by-line authorship of HTML, CSS, and JavaScript. Instead, a new methodology, colloquially termed "Vibe Coding," has emerged, centered on the

use of Large Language Models (LLMs) to generate, refine, and debug code through natural language prompts.² This evolution fundamentally reframes the development process on platforms like Wix. The challenge is no longer simply

how to code with Velo, but rather *how to architect a system that responsibly integrates AI as a development co-pilot*. This report serves as the essential bridge between the raw, accelerative potential of AI-generated code and the professional rigor required to engineer reliable, production-ready software. It outlines a disciplined approach that harnesses AI's speed while enforcing the architectural integrity, security, and performance standards demanded by scalable applications.

Report Structure and Objectives

This document is structured in three parts to provide a comprehensive guide from theory to implementation.

- **Part I: Deconstructing the "Vibe Coding" Paradigm for Professional Development** establishes the theoretical and practical foundation for using AI within a professional Wix/Velo workflow. It analyzes the core concepts of AI-assisted development, defines a spectrum of professional application, and outlines strategies for both leveraging its power and mitigating its inherent risks.
- **Part II: The Architectural Blueprint for a Standardized Wix MVP** translates this hybrid AI-Velo strategy into a concrete architectural framework. It provides the structural foundation of the reusable MVP template, covering code structure, responsive design, data persistence, performance optimization, and security.
- **Part III: Implementing Core MVP Functionality** delivers detailed, step-by-step implementation guides for the foundational features of the MVP template, including unified session management, custom member profiles, and a reusable module for tracking user history.

Part I: Deconstructing the "Vibe Coding" Paradigm for Professional Development

This section establishes the necessary conceptual framework for integrating AI into a professional development lifecycle. It moves beyond the hype of AI-generated code to define a disciplined, strategic approach that aligns with the goals of building robust and scalable

applications on the Wix platform.

1.1 Analysis of Tina Huang's "Vibe Coding Fundamentals"

Tina Huang's video, "Vibe Coding Fundamentals In 33 minutes," serves as a valuable entry point into this new paradigm, not as a technical tutorial for a specific language, but as a demonstration of a novel workflow.¹

Core Concepts:

Huang, an ex-Meta data scientist with a focus on AI and efficient learning, presents a methodology that prioritizes maximizing outcomes with minimal effort—a philosophy she directly attributes to her affinity for AI.⁴ Her workflow, while demonstrated on platforms like Replit, is platform-agnostic and introduces several key principles applicable to professional development 1:

1. **Start with a Plan:** The process begins not with code, but with a Product Requirements Document (PRD). This step grounds the project in clear objectives before any AI interaction occurs.¹
2. **Use Frameworks:** Leveraging existing frameworks provides structure and accelerates development, a principle that translates to using the Velo API framework within Wix.
3. **Version Control:** The use of GitHub for version control is emphasized as a non-negotiable step for managing iterations and maintaining a stable codebase, especially in an experimental, AI-driven process.¹
4. **The Core Loop:** The fundamental "vibe coding" loop is demonstrated as an iterative cycle: describe a high-level goal, have the AI generate code, execute and observe the result, and then provide feedback in natural language to refine or debug the output.¹
5. **Debugging:** The debugging process shifts from manual code inspection to feeding error messages back into the LLM, asking it to interpret and fix its own mistakes.¹

While Huang's examples do not involve Wix, the principles she outlines are directly transferable. The concept of guiding an AI to build an application based on high-level prompts aligns perfectly with the capabilities of Velo, particularly with the introduction of Wix's own native AI assistants designed to generate code snippets and assist developers.⁹

1.2 From "Vibes" to Verifiable Code: A Professional Spectrum

The term "Vibe Coding," popularized by AI researcher Andrej Karpathy, is not monolithic; it

exists on a spectrum of professional rigor.² Understanding this spectrum is critical for any team looking to adopt AI-assisted development for production-grade applications.

- **"Pure" Vibe Coding:** At one end of the spectrum lies the approach Karpathy described as "fully giving in to the vibes... and forgetting that the code even exists".³ In this model, the developer avoids examining the code, accepts AI suggestions without review, and focuses entirely on iterative experimentation. This approach is highly effective for rapid prototyping, building "software for one" (small, personalized tools), and learning unfamiliar technologies.³ However, its abdication of code ownership makes it inherently unsuitable for scalable, secure, and maintainable applications due to the high risk of introducing subtle bugs, security flaws, and unmanageable code.³
- **Responsible AI-Assisted Development:** This is the practical and professional application of the concept. Here, the AI tool functions as a powerful collaborator—a "pair programmer" or an advanced "typing assistant".² The developer remains the architect and quality gatekeeper. They guide the AI with precise prompts but then critically review, test, and refactor the generated code, taking full ownership of the final product. This model harnesses the speed and efficiency of AI for initial code generation while imposing the discipline and rigor of traditional software engineering. It is this model that forms the foundation of the framework proposed in this report.

The following table contrasts these two approaches to provide a clear decision-making framework.

Metric	"Pure" Vibe Coding	Responsible AI-Assisted Development
Development Speed	Extremely high for initial prototypes.	High, with overhead for review and testing.
Code Quality	Variable; often suboptimal and inconsistent.	High; standardized through manual review and refactoring.
Security Risk	High; vulnerabilities can be generated and go undetected.	Mitigated; code is reviewed for security flaws.
Maintainability	Low; lack of structure and developer understanding.	High; code is refactored to meet architectural standards.

Developer Role	Prompter and Tester.	Architect, Prompter, Reviewer, and Integrator.
Ideal Use Case	Rapid ideation, disposable prototypes, learning.	Production applications, feature development, complex systems.

1.3 Strategic Prompt Engineering for Velo

Karpathy's 2023 assertion that "the hottest new programming language is English" underscores a fundamental shift: the developer's skill is expanding from writing code to effectively instructing an AI to write code.³ Generic prompts produce generic and often incorrect or outdated code. A strategic, context-rich prompting methodology is required to generate useful Velo code.

A successful prompt for generating Velo code must provide the LLM with four layers of context¹⁴:

1. **Platform Declaration:** The prompt must begin by establishing the AI's persona and the target environment. For example: *"You are an expert Velo developer. All code you generate must be for the Wix.com platform and use the Velo APIs. The code will be written in JavaScript (ES2020)."* This immediately constrains the model to the correct ecosystem.¹⁴
2. **API Context:** LLMs often have outdated knowledge of APIs, which evolve rapidly.¹⁴ To ground the AI in the current reality, the prompt should include links to the specific Velo API documentation pages relevant to the task (e.g., `wix-data-v2.WixDataQuery`, `wix-members-backend.authentication`).
3. **UI & Data Context:** The AI has no knowledge of the specific site's structure. The prompt must explicitly list the element IDs (e.g., `#nameInput`, `#submitButton`, `#resultsRepeater`) and the exact collection field keys (e.g., `firstName`, `_owner`, `sessionData`) that the code will interact with.¹⁴
4. **Clear Logic:** The desired functionality should be described in a precise, step-by-step sequence of natural language commands. For example: *"When the #submitButton is clicked, get the value from #nameInput. Query the 'Members' collection where the 'lastName' field equals the input value. If results are found, populate the #resultsRepeater with the items. If no results are found, show the #noResultsText element."*

While external tools like ChatGPT are powerful, Wix provides its own native AI assistants. The

AI Assistant in the Wix IDE can generate and fix code snippets and look up documentation, offering a more integrated experience.⁹ The

AI Code Assistant in the Wix Studio Code Panel is more limited, designed to generate simple frontend event handlers, but it cannot handle backend logic or interactions with third-party services.¹⁹ A hybrid approach is therefore optimal: use the native assistants for quick, component-level tasks and a well-prompted external LLM for generating more complex, full-stack business logic.

1.4 Mitigating AI-Associated Risks

The acceleration offered by AI-assisted development is accompanied by significant risks that must be actively managed. AI-generated code can introduce subtle security vulnerabilities, suffer from poor performance, and be difficult to debug and maintain if not properly handled.¹² A non-developer asking an LLM for a program without specific constraints may receive code with critical flaws.²⁰

The "Responsible AI-Assisted Development" model requires a rigorous human-in-the-loop process to mitigate these dangers:

1. **Mandatory Code Review:** Every line of AI-generated code must be treated as if it were written by a junior developer. It requires a thorough review by a competent senior developer to check for logical errors, security anti-patterns (e.g., exposing sensitive data on the frontend), and adherence to project-specific best practices. This practice is the direct antithesis of the "pure vibe coding" ethos of not looking at the code.³
2. **Rigorous Testing and Monitoring:** The generated code must be validated using Velo's full suite of testing tools. This includes using the **Preview Mode** for immediate feedback, the **Developer Console** to check for logs and errors, and the **Monitoring Dashboard** to analyze the performance of backend requests and data queries on the live site.²¹
3. **Disciplined Version Control:** Wix's integration with Git is not optional in an AI-assisted workflow; it is essential.²¹ By creating branches for AI-generated features, developers can experiment safely, review changes in pull requests, and easily roll back if the new code introduces instability.
4. **Refactoring for Clarity and Maintainability:** AI models are optimized for functionality, not necessarily for elegance or long-term maintainability. A key role for the human developer is to refactor the AI's output. This may involve renaming variables for clarity, breaking large functions into smaller modules, and adding comments to explain complex logic, ensuring the code aligns with the project's architectural standards.²⁵

Part II: The Architectural Blueprint for a Standardized Wix MVP

This section translates the strategy of "Responsible AI-Assisted Development" into a concrete, reusable architectural framework for a Wix MVP. This blueprint provides the structural integrity necessary for building applications that are scalable, performant, and maintainable.

2.1 Foundation: Velo Code Structure and Best Practices

A well-organized codebase is the bedrock of a scalable application. This is especially true when integrating AI-generated code, which requires a clear structure to be placed into. The Velo environment provides a logical file system that should be used consistently across all projects.²³

- **File Organization:**
 - **Page Code:** Files in this section (src/pages/) contain JavaScript for a specific page. This code should be limited to UI logic, such as handling element events (.onClick(), .onChange()) and manipulating page elements.
 - **Public Files:** Public JavaScript files (src/public/) are accessible from any page on the frontend. They are ideal for housing utility functions (e.g., data formatters, custom animations) that are shared across multiple pages. No sensitive logic or API keys should ever be placed here.
 - **Backend Code:** Backend files (src/backend/) run on Wix's secure Node.js servers and are inaccessible from the browser. This is the *only* location for sensitive operations.
 - **Web Modules (.web.js):** These files expose functions that can be called from frontend code. This is the primary mechanism for secure communication between the client and server. All database queries, calls to third-party APIs, and business logic should be encapsulated in web module functions.
 - **Standard Modules (.js):** These are for backend logic that is not directly exposed to the frontend, such as internal helper functions or scheduled jobs (jobs.config).
- **Code Readability and Maintainability:** To ensure the template is easy to understand and extend, the following Velo best practices must be enforced²⁵:
 - **Meaningful Naming:** Element IDs must be descriptive (e.g., #userProfileImage, #loginButton). This not only makes the code self-documenting but is also a

prerequisite for writing effective AI prompts.

- **Modular Functions:** Avoid monolithic blocks of code. Decompose complex operations into smaller, single-responsibility functions. This improves testability and reusability.
- **Robust Error Handling:** Every asynchronous operation (e.g., data queries, backend calls) must be wrapped in a promise chain with a `.catch()` block to handle potential errors gracefully and provide feedback to the user, rather than crashing the application.
- **Consistent Formatting:** Utilize the built-in code formatter to maintain standard indentation and style, making the code easier to read for the entire team.
- **Maximizing Reusability with Custom Apps:** For organizations building multiple applications on Wix, the ultimate form of a "standard template" is a **Wix Custom App**.²⁷ This feature allows developers to package reusable functionality—including UI widgets, backend code, and even database collections—into a private, installable app. A core set of features, such as the custom authentication flow and user profile components detailed in this report, can be built as a Custom App and then installed on any new site with a single click, ensuring consistency and dramatically accelerating development.

2.2 The Responsive Core: Building for Every Device

Achieving true responsiveness across all devices requires moving beyond the basic Wix Editor and leveraging the advanced capabilities of **Wix Studio**, which should be considered the default environment for professional application development.²⁸

- **Key Techniques for a Fully Responsive Design:**
 - **Breakpoints:** Wix Studio provides default breakpoints for desktop, tablet, and mobile, but also allows for the creation of custom breakpoints. This enables precise control over how the layout adapts to different screen width ranges, ensuring an optimal viewing experience on all devices.²⁸
 - **Fluid Grids and Relative Units:** Instead of fixed pixel layouts, the architecture should employ fluid grids and relative units (like percentages and `fr` units). Features like CSS Grid and Flexbox, available in Wix Studio, allow elements to resize and reflow intelligently based on the available space, creating a truly adaptive design.³¹
 - **Responsive AI Tool:** Wix Studio includes a Responsive AI tool that can analyze a section's content and automatically apply the ideal layout tool (e.g., a stack for vertical elements, a grid for a gallery). This AI-generated layout serves as an excellent starting point that can then be manually fine-tuned by the developer, blending AI speed with human design oversight.¹¹
 - **Velo for Dynamic Adjustments:** For ultimate control, Velo's wix-window-frontend API can detect the user's device type (`formFactor`) and screen size. This allows for

programmatic adjustments that are impossible through the editor alone, such as loading a simplified version of a component on mobile or showing/hiding elements based on screen orientation.

2.3 Data Persistence and Performance

An efficient and scalable application is underpinned by a well-designed data schema and optimized data access patterns.

- **User Data Schema:** The MVP template will include a primary data collection, `MembersData`, with a schema designed for extensibility.
 - **Permissions:** Set to "Custom Use," with create permissions for "Site Member," and read/update permissions for "Site Member Author." This ensures users can create their own profile but can only read and modify their own data.³⁴
 - **Fields:**
 - `_id` (Text, Primary): Automatically linked to the Wix Member ID.
 - `firstName` (Text)
 - `lastName` (Text)
 - `profileImage` (Image)
 - `previousSessions` (Array of Objects): This field directly addresses the user requirement to store historical data. Each object in the array can represent a past result or session, containing fields like `sessionId`, `date`, and `sessionData`.
- **Wix Data API Optimization:** To ensure the application is fast and operates within Wix's platform limits, the following performance optimization techniques are mandatory³⁵:
 - **Query Optimization:** All data queries must be specific. Use filters (`.eq()`, `.contains()`) to retrieve only the required items and `.limit()` to paginate results, preventing the transfer of large, unnecessary datasets.³⁶
 - **Server-Side Rendering with `warmupData`:** This is a critical technique for improving perceived page load speed. By fetching the initial data needed for a page on the server during the rendering process, the results can be passed to the client as "warmup data." This prevents a second, redundant data query on the client-side, allowing the page to display content almost instantly.³⁷
 - **Web Method Caching:** For data that is frequently accessed but infrequently changed (e.g., a list of categories, public profiles), the return values of backend web module functions can be cached for a specified duration (Time To Live). Subsequent calls for the same data will be served instantly from the cache, reducing database load and improving API response times.³⁸
 - **Bulk Operations:** When writing multiple items to a collection (e.g., importing data, batch updates), always use bulk functions like `wixData.bulkInsert()` or `wixData.bulkUpdate()`. These operations count as a single data request, making them

far more efficient than iterating and performing individual writes.³⁵

2.4 Security Posture

While Wix provides an enterprise-grade security infrastructure, including a secure serverless Node.js environment, real-time threat monitoring, and compliance with standards like PCI DSS and SOC 2, application-level security remains the developer's responsibility.²¹

- **Core Security Principles for Velo Development:**
 - **Secure Web Modules:** All sensitive logic must be isolated in backend files (.js or .web.js). This includes all database interactions (queries, inserts, updates), calls to third-party APIs, and any logic that should not be exposed to the client's browser.
 - **Strict Collection Permissions:** As detailed in the data schema, collection permissions must be configured to the principle of least privilege. A user should only be able to create, read, and update their own data. This is enforced by leveraging the system-generated `_owner` field in queries and permission settings.
 - **Secrets Management:** API keys, tokens, and other credentials must never be hardcoded in frontend or even backend code. They must be stored using the Velo Secrets Manager. These secrets can then be safely accessed *only* from backend code, preventing their exposure.²¹
 - **Backend Input Validation:** Never trust data coming from the client. Before any data received from a frontend form is written to the database, it must be validated and sanitized on the backend to prevent common vulnerabilities like Cross-Site Scripting (XSS) and data corruption.

Part III: Implementing Core MVP Functionality

This section provides detailed implementation guides for the foundational features of the MVP template. These guides serve as a practical application of the architectural principles and AI-assisted workflow established in the previous sections, offering concrete code strategies and API usage patterns.

3.1 Unified Session Management for Cross-Device Access

A central requirement for a modern application is the ability for a user to maintain a consistent experience across multiple devices. Standard browser-based sessions are inherently tied to a single device. To achieve a unified, cross-device session state, a custom, token-based authentication flow must be implemented. This architecture ensures that when a user logs in on any device, they are connected to the same central user account, granting them access to their persistent data, such as previous session results.

The solution involves a precise sequence of frontend and backend API calls:

1. **Custom UI:** Build custom lightbox or page-based forms for user registration and login. This provides full control over the user experience, moving away from Wix's default forms.³⁹
2. **Backend Credential Verification:** When a user submits the login form, the frontend does not perform the login itself. Instead, it securely calls a web module function in the backend (e.g., `loginUser(email, password)`). This backend function uses `wix-members-backend.authentication.login()` to verify the user's credentials against the Wix Members database. This ensures the user's password is never exposed on the client side.⁴²
3. **Backend Session Token Generation:** Upon successful credential verification, the backend immediately calls `wix-members-backend.authentication.generateSessionToken(email)`. This function creates a secure, single-use token that is authorized to log in that specific user.⁴⁴ This token is the critical component for enabling the cross-device session.
4. **Frontend Session Application:** The backend function returns the generated session token to the frontend. The frontend code then calls `wix-members-frontend.authentication.applySessionToken(sessionToken)`, which consumes the token and establishes a valid, logged-in session for the user on that specific device's browser.⁴⁶

When the user moves to a new device and logs in, this entire flow is repeated. A new session token is generated and applied, logging them into the same underlying account. Because all user data and history are stored centrally in a Wix Data collection linked to their unique member ID, the experience is seamless and consistent, regardless of the device used.

The following table outlines the key Velo APIs involved in this secure, cross-device authentication workflow.

API Module	Function	Purpose in Workflow	Code Context

wix-members-back end	authentication.login() ()	Verifies user credentials securely on the server.	Backend
wix-members-back end	authentication.generateSessionToken() ()	Creates a single-use token to initiate a session for a verified user.	Backend
wix-members-front end	authentication.applySessionToken() ()	Consumes the session token to log the user in on the client device.	Frontend
wix-members-front end	currentUser.getMember() ()	Retrieves the full member object after a session is established.	Frontend

3.2 Building the Custom Member Profile Area

The custom member profile area is a cornerstone of the MVP template, demonstrating a data-driven, secure, and dynamic user experience. This implementation synthesizes best practices from multiple Velo tutorials to create a robust and extensible module.³⁴

Implementation Steps:

1. **Create the MembersData Collection:** In the Wix CMS, create a new collection with the schema defined in Part II. Critically, set the collection permissions to "Custom Use," allowing a "Site Member" to create content and a "Site Member Author" to read and update their own content. This permission model is the foundation of the profile's security.³⁴
2. **Create Dynamic Pages:**
 - Create a dynamic item page with a URL structure like Members/{ID}. This will be the main profile view page.
 - Create a second dynamic item page with a URL like Members/Update/{ID}. This will be the profile editing page.

- Set the page permissions for both dynamic pages to "Members Only" to restrict access to logged-in users.
3. **Configure Datasets and UI:**
- On the profile view page (Members/{ID}), add a dataset connected to the MembersData collection and set its mode to **Read-only**. Add text elements to the page and connect them to the corresponding fields in the dataset to display the user's profile information.
 - On the profile edit page (Members/Update/{ID}), add a dataset connected to the MembersData collection and set its mode to **Read & Write**. Add user input elements (text inputs, image uploads, etc.) and connect them to the dataset fields. Add a "Submit" button and configure its click action to "Submit" in the dataset connection panel.
4. **Implement Navigation and Data Flow with Velo:**
- On the profile view page, add an "Edit Profile" button. In the page's onReady function, get the current user's ID and write an .onClick() event handler for the button that navigates them to their unique update page using wixLocation.to().

JavaScript

// In onReady() function on the Profile View page

```
import wixLocation from 'wix-location-frontend';
```

```
import { currentUser } from 'wix-members-frontend';
```

```
let memberId;
```

```
currentUser.getMember({ fieldsets: ['FULL'] })
```

```
.then((member) => {
```

```
  memberId = member._id;
```

```
});
```

```
$w('#editProfileButton').onClick(() => {
```

```
  wixLocation.to(`/Members/Update/${memberId}`);
```

```
});
```

- The dataset on the update page will handle the data submission automatically. Upon successful submission, you can configure the dataset to navigate the user back to their profile view page.

3.3 Template in Action: A Reusable "Previous Sessions" Module

To provide a concrete example that directly fulfills the requirement to "access previous Results/sessions," this section details the creation of a reusable module for storing and

displaying user history.

1. **Extend the Data Schema:** Modify the MembersData collection by adding a new field named sessions. Set its type to **Array**. This array will store objects, each representing a past session.
2. **Create a Backend Web Module (sessions.web.js):** This module will contain the secure logic for managing session data.

JavaScript

```
// In src/backend/sessions.web.js
```

```
import wixData from 'wix-data';
```

```
import { currentMember } from 'wix-members-backend';
```

```
export async function addSession(sessionData) {
```

```
  const memberId = await currentMember.getMember().then(member => member._id);
```

```
  if (!memberId) {
```

```
    throw new Error("Member not logged in.");
```

```
  }
```

```
  // Get the current sessions array
```

```
  const memberItem = await wixData.get("MembersData", memberId);
```

```
  const currentSessions = memberItem.sessions ||;
```

```
  // Add the new session
```

```
  const newSession = {
```

```
    _id: // generate a unique ID if needed
```

```
    date: new Date(),
```

```
    data: sessionData
```

```
  };
```

```
  currentSessions.push(newSession);
```

```
  // Update the member's item in the collection
```

```
  const toUpdate = {...memberItem, sessions: currentSessions};
```

```
  return wixData.update("MembersData", toUpdate);
```

```
}
```

```
export async function getSessions() {
```

```
  const memberId = await currentMember.getMember().then(member => member._id);
```

```
  if (!memberId) {
```

```
    return;
```

```
  }
```

```
const memberItem = await wixData.get("MembersData", memberId);
return memberItem.sessions ||;
}
```

3. Frontend Implementation:

- **Saving a Session:** On any page where a session is completed (e.g., after a quiz), import and call the addSession function from the backend module.

JavaScript

// On a quiz results page

```
import { addSession } from 'backend/sessions.web.js';
```

```
const quizResult = { score: 95, topic: "History" };
```

```
addSession(quizResult)
.then(() => {
  console.log("Session saved successfully.");
})
.catch(error => {
  console.error("Failed to save session:", error);
});
```

- **Displaying Sessions:** On the member profile page, add a repeater element (#sessionsRepeater). In the page's onReady function, call the getSessions function and use the results to populate the repeater.

JavaScript

// In onReady() on the Profile View page

```
import { getSessions } from 'backend/sessions.web.js';
```

```
$w.onReady(function () {
  //... other onReady code...
```

```
  getSessions()
  .then(sessions => {
    $w('#sessionsRepeater').data = sessions;
  });
});
```

```
$w('#sessionsRepeater').onItemReady(($item, itemData, index) => {
  $item('#sessionDateText').text = itemData.date.toLocaleDateString();
  $item('#sessionScoreText').text = `Score: ${itemData.data.score}`;
});
```

This modular approach creates a fully functional, secure, and reusable system for tracking and displaying user history, directly fulfilling a core requirement of the MVP template.

Conclusion and Strategic Recommendations

The development of a standardized MVP template on the Wix platform requires a strategic synthesis of architectural best practices, advanced platform features, and the disciplined application of emerging AI-assisted workflows. The framework outlined in this report provides a replicable blueprint for achieving this synthesis, enabling the rapid development of applications that are not only functional at launch but are also engineered for scalability, performance, and long-term maintainability.

The "Day 1 Standard Template" Checklist

This checklist serves as a concise, actionable guide for initiating any new application project using this framework.

- **1. Environment Setup:**
 - ☐ **Select Wix Studio:** Utilize Wix Studio as the primary editor to access advanced responsive design and professional development features.
 - ☐ **Enable Coding:** Ensure the coding environment is active to access the Code Panel and Velo APIs.
 - ☐ **Integrate Git:** Initialize a Git repository for the site to enable version control from day one.
- **2. Code and Data Architecture:**
 - ☐ **Establish File Structure:** Create placeholder files in Public for shared utilities and in Backend for initial web modules (e.g., members.web.js, sessions.web.js).
 - ☐ **Create MembersData Collection:** Set up the core user data collection in the CMS.
 - ☐ **Configure Collection Permissions:** Immediately set the MembersData collection permissions to "Custom Use" with "Site Member Author" restrictions to secure user data.
- **3. Core Pages and Components:**
 - ☐ **Build Custom Login/Registration:** Create a lightbox or page for custom user authentication.
 - ☐ **Create Dynamic Profile Pages:** Set up the Members/{ID} (view) and

Members/Update/{ID} (edit) dynamic pages.

- [] **Set Page Permissions:** Ensure all member-specific pages are set to "Members Only."
- **4. Essential Code Implementation:**
 - [] **Implement Token-Based Login:** Write and test the frontend and backend code for the secure, cross-device session token flow.
 - [] **Code Profile Navigation:** Implement the Velo code for navigating between the view and update profile pages.
 - [] **Implement a Base Module:** Code the initial version of a reusable module, such as the "Previous Sessions" component, to validate the architecture.
- **5. AI-Assisted Workflow:**
 - [] **Develop Prompt Templates:** Create a set of standardized prompt templates for generating Velo code that includes platform, API, and UI context.
 - [] **Enforce Review Process:** Establish a mandatory code review step for all AI-generated code before it is merged into the main branch.

The Future of AI in Low-Code Development

The "Vibe Coding" paradigm represents a fundamental evolution in the role of the software developer. As AI models become increasingly capable of generating complex, functional code from natural language prompts, the developer's primary value shifts away from the rote transcription of logic into syntax. Instead, the developer is elevated to the role of **system architect, strategic prompter, and rigorous quality assurance engineer**. Their expertise becomes crucial in designing the overarching structure, providing the precise context and constraints for the AI, and, most importantly, validating the output for security, performance, and correctness. In the context of a platform like Wix, this means the most effective developers will be those who can master both the Velo APIs and the art of instructing an AI, using the platform not just as a visual builder but as a powerful, integrated development environment where they direct both human and machine intelligence.

Final Recommendations

To successfully build scalable applications using this framework, it is essential to strike a strategic balance between the velocity offered by AI and the rigor of professional engineering.

- **Utilize AI for Acceleration:** Employ the AI-assisted workflow heavily for tasks such as scaffolding new components, generating boilerplate code for UI elements, implementing

standard algorithms, and creating initial drafts of data queries. This is where AI provides the greatest return on investment in terms of speed.

- **Reserve Manual Craftsmanship for Critical Systems:** For mission-critical components of the application, a more traditional, manual coding and review process remains essential. This applies specifically to:
 - **Authentication and Security Logic:** The custom session token flow and any code handling sensitive data should be meticulously hand-coded and reviewed.
 - **Performance-Sensitive Queries:** Complex database queries that could impact site performance should be manually optimized.
 - **Core Architectural Modules:** The foundational backend web modules that define the application's core logic should be engineered with human oversight to ensure long-term maintainability.

By adopting this hybrid approach—leveraging AI as a powerful co-pilot while retaining human architectural oversight and quality control—development teams can build better products faster, transforming the Wix platform into a truly professional-grade application development ecosystem.

Works cited

1. Vibe Coding Fundamentals In 33 minutes - YouTube, accessed September 10, 2025, <https://www.youtube.com/watch?v=iLCDSY2XX7E>
2. Vibe Coding Explained: Tools and Guides | Google Cloud, accessed September 10, 2025, <https://cloud.google.com/discover/what-is-vibe-coding>
3. Vibe coding - Wikipedia, accessed September 10, 2025, https://en.wikipedia.org/wiki/Vibe_coding
4. Tina Huang - YouTube, accessed September 10, 2025, https://www.youtube.com/channel/UC2UXDak6o7rBm23k3Vv5dwww/videos?view=0&sort=dd&shelf_id=0
5. Good coding habits to start developing NOW - YouTube, accessed September 10, 2025, <https://www.youtube.com/watch?v=YUQ18uSrwqE>
6. Tina Huang Podcast Republic, accessed September 10, 2025, <https://podcastrepublic.net/podcast/1811026238>
7. Top Tech Women on YouTube to Follow for AI Insights in 2025 - Lemon8-app, accessed September 10, 2025, <https://www.lemon8-app.com/@peachie.suga/7518363693441090061?region=us>
8. How To Get The Most Out Of Vibe Coding | Startup School - YTScribe, accessed September 10, 2025, <https://ytscribe.com/v/BJjsfNO5JTo>
9. Generate Code Using the Wix AI Assistant - Wix Developers, accessed September 10, 2025, <https://dev.wix.com/docs/develop-websites/articles/workspace-tools/developer-tools/ai-assistants/ai-assistant-in-the-wix-ide/generate-code-using-the-wix-ai-assistant>
10. About the Wix AI Assistant - Wix Developers, accessed September 10, 2025, <https://dev.wix.com/docs/develop-websites/articles/workspace-tools/developer-t>

- [ools/ai-assistants/ai-assistant-in-the-wix-ide/about-the-wix-ai-assistant](#)
11. AI Tools That Transform How Agencies Work | Wix Studio, accessed September 10, 2025, <https://www.wix.com/studio/ai/blank-2>
 12. What is Vibe Coding? | IBM, accessed September 10, 2025, <https://www.ibm.com/think/topics/vibe-coding>
 13. What is vibe coding? | AI coding | Cloudflare, accessed September 10, 2025, <https://www.cloudflare.com/learning/ai/ai-vibe-coding/>
 14. Why is coding with AI so hard! : r/WIX - Reddit, accessed September 10, 2025, https://www.reddit.com/r/WIX/comments/1liopta/why_is_coding_with_ai_so_hard/
 15. How to use ChatGPT to write code - and my top trick for debugging what it generates, accessed September 10, 2025, <https://www.zdnet.com/article/how-to-use-chatgpt-to-write-code-and-my-top-trick-for-debugging-what-it-generates/>
 16. Anyone else use ChatGPT to write python code to analyze their .Fit Files : r/Velo - Reddit, accessed September 10, 2025, https://www.reddit.com/r/Velo/comments/1l1899p/anyone_else_use_chatgpt_to_write_python_code_to/
 17. Can ChatGPT Write Velo Code? - YouTube, accessed September 10, 2025, <https://www.youtube.com/watch?v=PGn-BgyO2Fs>
 18. With Velo code, is it possible to add new fields (including text) to items in a database?, accessed September 10, 2025, <https://forum.wixstudio.com/t/with-velo-code-is-it-possible-to-add-new-fields-including-text-to-items-in-a-database/56303>
 19. About the AI Code Assistant - Wix Developers, accessed September 10, 2025, <https://dev.wix.com/docs/develop-websites/articles/workspace-tools/developer-tools/ai-assistants/ai-assistant-in-the-code-panel/about-the-ai-code-assistant>
 20. I tried "vibe coding" with ChatGPT, and the vulnerabilities made me never want to use it again - XDA Developers, accessed September 10, 2025, <https://www.xda-developers.com/tried-vibe-coding-chatgpt-vulnerabilities/>
 21. Website development with a full stack of APIs and tools | Wix Studio, accessed September 10, 2025, <https://www.wix.com/studio/developers/web-development>
 22. Working with the Monitoring Dashboard - Wix Developers, accessed September 10, 2025, <https://dev.wix.com/docs/develop-websites/articles/workspace-tools/testing-monitoring/monitoring-your-published-site/working-with-the-monitoring-dashboard>
 23. Getting Oriented - Wix Developers, accessed September 10, 2025, <https://dev.wix.com/docs/velo/articles/getting-started/getting-oriented>
 24. About Automated Workflows and the Wix CLI - Wix Developers, accessed September 10, 2025, <https://dev.wix.com/docs/develop-websites/articles/workspace-tools/developer-tools/git-integration-wix-cli/about-automated-workflows-and-the-wix-cli>
 25. Best Practices for Building a Site with Velo - Wix Developers, accessed September 10, 2025, <https://dev.wix.com/docs/develop-websites/articles/best-practices/best-practices-for-building-a-site-with-velo>

26. About Coding with Wix Studio - Wix Developers, accessed September 10, 2025, <https://dev.wix.com/docs/develop-websites/articles/coding-with-velo/overview/about-coding-with-wix-studio>
27. About Custom Apps - Wix Developers, accessed September 10, 2025, <https://dev.wix.com/docs/develop-websites/articles/coding-with-velo/packages/about-custom-apps>
28. Creating Responsive Sites With Wix | Help Center, accessed September 10, 2025, <https://support.wix.com/en/article/creating-responsive-sites-with-wix>
29. Studio Editor: Building a Responsive Site | Help Center | Wix.com, accessed September 10, 2025, <https://support.wix.com/en/article/studio-editor-building-a-responsive-site>
30. Wix Studio vs Wix AI website builder: What's the difference? - TechRadar, accessed September 10, 2025, <https://www.techradar.com/pro/website-building/wix-studio-vs-wix-ai>
31. How to make a responsive website in 4 steps - Wix.com, accessed September 10, 2025, <https://www.wix.com/blog/how-to-make-a-responsive-website>
32. The ultimate guide to responsive website design - Wix.com, accessed September 10, 2025, <https://www.wix.com/blog/responsive-website-design>
33. Responsive design problems - Ask the community - Wix Studio Forum, accessed September 10, 2025, <https://forum.wixstudio.com/t/responsive-design-problems/69864>
34. Building Your Own Members Area - Wix Developers, accessed September 10, 2025, <https://support.wix.com/en/article/velo-tutorial-building-your-own-members-area>
35. Optimizing Your Code - Wix Developers, accessed September 10, 2025, <https://dev.wix.com/docs/develop-websites/articles/coding-with-velo/limits-and-optimization/optimizing-your-code>
36. Best Practices for Improving Performance in Wix Sites with Data, accessed September 10, 2025, <https://dev.wix.com/docs/develop-websites/articles/best-practices/best-practices-for-improving-performance-in-wix-sites-with-data>
37. Optimize Page Rendering with Warmup Data - Wix Developers, accessed September 10, 2025, <https://dev.wix.com/docs/develop-websites/articles/coding-with-velo/frontend-code/page-rendering/optimize-page-rendering-with-warmup-data>
38. About Web Method Caching - Wix Developers, accessed September 10, 2025, <https://dev.wix.com/docs/develop-websites/articles/best-practices/caching/about-web-method-caching>
39. Creating a Custom Registration Form with Code - Wix Developers, accessed September 10, 2025, <https://dev.wix.com/docs/develop-websites/articles/code-tutorials/wix-members/creating-a-custom-registration-form-with-code>
40. Custom Registration form - Ask the community - Wix Studio Forum, accessed September 10, 2025,

- <https://forum.wixstudio.com/t/custom-registration-form/36638>
41. How To use Wix Code to Create a Custom Member Signup and Login Experience | Velo, accessed September 10, 2025,
https://www.youtube.com/watch?v=k_OxbNS89XY
 42. Login | Velo, accessed September 10, 2025,
<https://dev.wix.com/docs/velo/apis/wix-members-backend/authentication/login>
 43. New Article: How to Create Member Profile Pages - Show and tell - Wix Studio Forum, accessed September 10, 2025,
<https://forum.wixstudio.com/t/new-article-how-to-create-member-profile-pages/28548>
 44. Generate Session Token | Velo, accessed September 10, 2025,
<https://dev.wix.com/docs/velo/apis/wix-members-backend/authentication/generate-session-token>
 45. import { authentication } from 'wix-members-backend' - miniOrange, accessed September 10, 2025, <https://www.miniorange.com/downloads/backend.jsw.pdf>
 46. Apply Session Token | Velo, accessed September 10, 2025,
<https://dev.wix.com/docs/velo/apis/wix-members-frontend/authentication/apply-session-token>
 47. Building Your Own Members Area - Wix Developers, accessed September 10, 2025,
<https://dev.wix.com/docs/develop-websites/articles/code-tutorials/wix-members/building-your-own-members-area>
 48. How to implement "member profile pages" on my site? - Wix Studio Forum, accessed September 10, 2025,
<https://forum.wixstudio.com/t/how-to-implement-member-profile-pages-on-my-site/26143>
 49. CMS: Creating Custom Member Profile Pages | Help Center | Wix.com, accessed September 10, 2025,
<https://support.wix.com/en/article/cms-creating-custom-member-profile-pages>
 50. Customizing Members Area with Velo - Educative.io, accessed September 10, 2025,
<https://www.educative.io/courses/build-membership-with-velo-wix/customizing-members-area-with-velo>